

Uncertainty-gated selection for block-sparse attention*

A value-of-information view of the SSA-style selector

Thomas Rossi
Eonpass
t.rossi@eonpass.com

June 13, 2026

Abstract

Block-sparse attention scales long-context language models by replacing the $O(N^2)$ softmax with a per-query top- k selection over key blocks. This cutoff is myopic: when the k -th and $(k+1)$ -th blocks are nearly tied in score, the selector commits without spending extra budget, and a dropped block carrying answer evidence is unrecoverable downstream. We propose a value-of-information *router* that measures, for each query, how decisively the top- k cut was made, and doubles the kept set for the queries where that gap is smallest; the rule is backbone-agnostic and stacks with existing block-scoring methods such as Quest. On LongBench-v2 medium at $n = 215$ (the entire dataset subset), router-on-Quest reaches paired recall 0.75 vs. top- k 0.47 – +28 pp over the SSA-style baseline (McNemar $p < 0.01$) – while attending to $\sim 4\%$ of the context; it lands within 2 pp of dense on RULER NIAH multikey at the same context. The lift reproduces on three models from two architecture families and at 128K context, where router-on-Quest holds paired recall 0.81 while the SSA-style baseline collapses to 0.09. The fused selection-plus-kernel pipeline runs at $0.57\times$ dense wall time at 128K.

1 Introduction

Long-context language models increasingly use *block-sparse attention* as a drop-in replacement for the $O(N^2)$ softmax. The idea is shared by Quest (Tang et al., 2024), H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), MInference (Jiang et al., 2024), NSA (Yuan et al., 2025), MoBA (Lu et al., 2025), and Subquadratic’s SSA (Subquadratic, 2025): a cheap per-query *selector* picks k out of N_B key blocks; exact attention then runs only on the selected set. The selector is the lever – it decides where the model attends. Today it is universally implemented as a *single-shot top- k rule* on a pooled inner product $q \cdot \bar{k}_b$ between the query and a per-block key summary.

This is a myopic decision. When the k -th and $(k+1)$ -th block are nearly tied, top- k silently breaks the tie and moves on; if the dropped block contained an evidence token, the answer is gone, and no downstream layer can recover it. The failure mode is structural, not noisy: it bites hardest on *multi-hop* and *query-latent* retrieval, where the relevance of a block depends on what was learned earlier in the same forward pass and is not visible to the selector’s surface query–key match. SSA’s own reported NIAH multi-key recall degrades sharply as the number of keys grows (Subquadratic, 2025).

*Code, data, and reproduction scripts: <https://github.com/ThomasRossi/uncertainty-gated-block-sparse-attention>. Persistent archive: [doi:10.5281/zenodo.20630587](https://doi.org/10.5281/zenodo.20630587) (concept DOI; always resolves to the latest version).

This paper. We treat the top- k cutoff as a *value-of-information* (VoI) decision and add a single layer of policy on top of it. For each Q-tile and head, we compute the normalised cutoff margin

$$\sigma = \frac{s_{(k-1)} - s_{(k)}}{s_{(0)} - s_{(k)}} \in [0, 1],$$

where $s_{(\cdot)}$ are the sorted block scores. A small σ means the cutoff is high-risk; we then *route* that tile to a $2\times$ expanded `kv_idx` – it gets to attend to more blocks – while confident tiles keep the baseline k_{budget} . The expansion is paid for selectively: we trigger on the bottom q -fraction of tiles per layer, so the average attended set grows by only $1 + q$ blocks per row.

The router is backbone-agnostic. The cutoff margin σ is a function of the sorted block scores; it does not depend on how those scores are computed. Existing selectors differ in their block-scoring backbone – SSA pools keys with a mean ($\bar{k}_b = \frac{1}{B_n} \sum_j k_j$), Quest uses a min/max upper bound ($s = \sum_d \max(q_d k_{b,d}^{\max}, q_d k_{b,d}^{\min})$). The router can sit on top of any of them. This turns the contribution from a replacement for top- k into a *generic budget-allocation layer that stacks on whichever scoring backbone is best for the task*. We validate this empirically: **better scoring (Quest) and better budget allocation (router) are orthogonal directions; combining them strictly dominates either alone** on both benchmarks we test.

Contributions.

1. A VoI formulation of the selector cutoff that adds one scalar per tile and one quantile threshold per layer, independent of the block-scoring backbone. No retraining, no extra parameters, no per-row keep tensor.
2. Empirical demonstration that the router composes with two distinct scoring backbones (SSA-style K-mean and Quest’s K-min/K-max) and that the combination strictly dominates either alone. The result holds on two standardised benchmarks (RULER NIAH multi-key and LongBench-v2 medium), three models from two architecture families (Qwen2.5-14B-Instruct, Qwen2.5-7B-Instruct-1M, Mistral-Nemo-Instruct-2407), and contexts from 32K to 128K. Headline: router-on-Quest paired recall **0.75** vs top- k 0.47 on LongBench-v2 medium at $n = 215$ (+28 pp, McNemar $p < 0.01$). Full panel in Section 4.
3. A fused selection-plus-kernel implementation that produces `kv_idx` directly per Q-tile and handles the routed expansion in the same code path, keeping kernel dispatch shape-uniform. All four sparse policies (top- k , router, Quest, router-on-Quest) run on this same kernel; they differ only in the selection step. Wall-time profile crosses dense at 32K and widens at 64K ($0.76\times$ – $0.85\times$); the crossover regime is characterised via an Amdahl decomposition of the prefill.
4. A custom diagnostic benchmark, the *Pointer-Chase Haystack* (PCH; Appendix B), used during method development to isolate selector quality from model capability.
5. A negative-control result (LongBench-v1) that pins down *when* the router helps: only when the selector’s per-query budget (the blocks it keeps) is small relative to where the answer-relevant evidence lives.

2 Background and related work

2.1 Block-sparse attention

A standard decoder layer (Vaswani et al., 2017) maps hidden state $h^{(L)} \in \mathbb{R}^{N \times d_{\text{model}}}$ to $h^{(L+1)}$ via

$$h' = h^{(L)} + \text{Attn}(\text{LN}(h^{(L)})), \quad h^{(L+1)} = h' + \text{MLP}(\text{LN}(h')).$$

The attention block, for query position i on head h , computes

$$\text{Attn}(x)_{i,h} = \sum_{j \leq i} \frac{\exp(q_{i,h} \cdot k_{j,h} / \sqrt{d_{\text{head}}})}{\sum_{j' \leq i} \exp(q_{i,h} \cdot k_{j',h} / \sqrt{d_{\text{head}}})} v_{j,h}.$$

Block-sparse attention replaces $\{j \leq i\}$ by a small selected subset S_i , chosen per query at inference time on frozen weights.

2.2 The selector landscape

Concrete selectors differ in (a) how they pool keys into a per-block summary and (b) how they score blocks against the query, but **they all reduce, at the per-query selection step, to a top- k rule over a block score**:

- **SSA** (Subquadratic, 2025) (Subquadratic Sparse Attention) – per-Q-tile top- k over mean-pooled keys, $s_b = q \cdot \bar{k}_b$ with $\bar{k}_b = \frac{1}{B_n} \sum_j k_j$. Cheap to compute; mean-pooling blurs single hot keys, which is exactly the multi-key NIAH degradation discussed above.
- **Quest** (Tang et al., 2024) – per-block elementwise $K_b^{\min}, K_b^{\max}$ summaries; score is an upper bound on $\max_{j \in b} q \cdot k_j$, computed coordinate-wise (see Eq. 2). Recovers sharp single-key needles that mean-pooling loses.
- **H2O** (Zhang et al., 2023), **SnapKV** (Li et al., 2024), **MInference** (Jiang et al., 2024) – top- k over learned or attention-history-based block scores.
- **NSA** (Yuan et al., 2025), **MoBA** (Lu et al., 2025) – end-to-end learned gating, but still resolves to a top- k over block scores at the per-query selection step.

What none of these methods do is treat the cutoff as a *decision under uncertainty*: when $s_{(k-1)} \approx s_{(k)}$, the selector commits without spending extra budget. That step is the lever this paper pulls. Because the lever is on the cutoff (not on how s is computed), it composes with any of the scoring backbones above. In the experiments we evaluate it on top of both the SSA-style K-mean backbone and Quest’s K-max upper bound.

2.3 Long-context evaluation

The published benchmarks fall into two families. **Synthetic / diagnostic**: RULER (Hsieh et al., 2024) (NIAH, VT), BABILong, MRCR. Designed for controlled stress tests of long-context recall. Their failure modes are interpretable but they are *not* predictive of downstream performance, as HELMET (Yen et al., 2024) has documented. **Real-task**: LongBench (Bai et al., 2023, 2024), HELMET (Yen et al., 2024), NoCha. These cover multi-hop QA, summarisation, code, and so on. LongBench-v2 in particular has a *medium* split with native prompt lengths typically $\geq 100\text{K}$ words, designed to stress long-context selection. We use **RULER NIAH** (synthetic, standardised) and **LongBench-v1 + v2 medium** (real-task, standardised) as the headline benchmarks, with a custom diagnostic benchmark (**PCH**, Appendix B) used during method development.

3 Method

We describe the working method end-to-end. Each step is motivated by the previous one and lifts a specific weakness. The full per-equation derivation lives in Appendix A; here we give the path the

reader needs to follow to understand the experiments.

3.1 Step 1: block scoring

Keys are grouped into contiguous blocks of $\text{BLOCK}_N = 64$ tokens. We evaluate the router on top of two block-scoring rules.

The first is the SSA-style mean-pooled-key inner product: at layer L , the selector scores block $b \in \{0, \dots, N_B - 1\}$ via

$$s^{(L)}[i, h, b] = \frac{q_i^{(L,h)} \cdot \bar{k}_b^{(L,h)}}{\sqrt{d_{\text{head}}}}, \quad \bar{k}_b^{(L,h)} = \frac{1}{\text{BLOCK}_N} \sum_{j \in \text{block } b} k_j^{(L,h)}. \quad (1)$$

Mean pooling blurs single-key signals: a block containing one hot key adjacent to noise scores like a block of all noise, and gets dropped.

The second is Quest’s K-min/K-max upper bound (Tang et al., 2024), which replaces $\bar{k}_b$ with an elementwise pair $(K_b^{\min}, K_b^{\max})$ and scores

$$s_b^{\text{quest}} = \sum_{d=1}^{d_{\text{head}}} \max(q_d \cdot K_{b,d}^{\max}, q_d \cdot K_{b,d}^{\min}), \quad (2)$$

a coordinate-wise upper bound on $\max_{j \in b} q \cdot k_j$ that preserves the single-key signals the mean averages away.

We treat the choice between Eqs. (1) and (2) as a *backbone hyperparameter*; everything downstream (per-tile selection, the cutoff margin σ , the trigger) is identical.

3.2 Step 2: per-tile selection

A naive per-row top- k emits a boolean keep tensor of shape $[B, H, M, N_B]$ that downstream must sort and union across rows of a Q-tile; this becomes the dominant cost at long context.

Following the SSA recipe, we group $\text{BLOCK}_M = 64$ consecutive query rows into a *Q-tile* t and make one selection decision per tile, shared by all rows in that tile:

$$\text{tile_score}[t, h, b] = \max_{r \in \text{tile } t} s[r, h, b], \quad \text{kv_idx}[t, h] = \text{top-}k(\text{tile_score}[t, h, \cdot]). \quad (3)$$

The sink block 0 and the tile’s own block are forced into kv_idx by adding $+\infty$ to their scores *before* the top- k , which avoids a downstream concat-and-dedup. The output is a single integer tensor of shape $[B, H, Q_t, k_{\text{budget}}]$ (where $Q_t = M/\text{BLOCK}_M$), passed straight to the attention kernel without a per-row inner mask. Wall time for selection scales as $O(NN_B/\text{BLOCK}_M)$ rather than $O(NN_B)$ for the per-row version.

This is coarser than per-row by construction: a block that one row in the tile strongly wants can be outranked by a block that several rows weakly want, since the tile score is a max over rows of an inner product. The router (Steps 3–6) addresses this – it spends a controlled amount of extra budget on the tiles where the top- k cut was ambiguous, while leaving confident tiles untouched.

3.3 Step 3: the cutoff is a decision – read its uncertainty

The top- k in Eq. (3) is a decision: keep block $(k-1)$, drop block (k) . Its *quality* is naturally measured by how decisive that ranking is. Let $s_{(0)} \geq s_{(1)} \geq \dots$ be the sorted tile scores for tile t , head h . We define the normalised cutoff margin

$$\sigma[t, h] = \frac{s_{(k-1)} - s_{(k)}}{s_{(0)} - s_{(k)}} \in [0, 1]. \quad (4)$$

The interpretation is direct:

- $\sigma \rightarrow 1$ – the kept set is far above the rejected tail; the cutoff is unambiguous.
- $\sigma \rightarrow 0$ – the kept set’s last element is tied with the first rejected element; the cutoff is a coin flip.

σ is computed from a top- $(k+1)$ partial sort – one extra element beyond the top- k that produces kv_idx – at no extra asymptotic cost.

Why this is a value-of-information signal. If we were to commit to the top- k , the expected loss from the cutoff decision is bounded by the probability mass the softmax would have placed on the dropped block. When the k -th and $(k+1)$ -th scores are close, that mass is large; when they are well-separated, it is small. The cutoff margin σ is a cheap, dimensionless proxy for this expected loss. The formal anchor is in Appendix D: under a sub-Gaussian noise model on the block scores, σ is the dimensionless analogue of the asymptotically optimal best-arm-identification exploration index of Garivier and Kaufmann (2016), and the σ -quantile router is $C(\tau)$ -times optimal in top- k identification, with $C(\tau) \leq 2$ under Gaussian noise.

3.4 Step 4: aggregate σ across heads

We aggregate the per-head margin $\sigma[t, h]$ to a per-tile signal via a uniform mean:

$$\bar{\sigma}[t] = \frac{1}{H} \sum_{h=1}^H \sigma[t, h]. \quad (5)$$

$\bar{\sigma}[t]$ is the tile’s overall decision confidence: low when many heads simultaneously face an ambiguous cutoff, high when most heads agree the cutoff is clean.

Aggregation is necessary: gating directly on the per-cell $\sigma[t, h]$ does not work (Appendix C). Heads in the same tile are highly correlated, and per-cell gating is dominated by per-head idiosyncratic noise that has no relation to whether the tile is actually ambiguous. Averaging across heads recovers the signal.

3.5 Step 5: trigger on the bottom q -fraction

We turn $\bar{\sigma}$ into a binary decision via a per-layer empirical quantile:

$$\text{trigger}[t] = \mathbb{I}[\bar{\sigma}[t] \leq \text{quantile}_q(\bar{\sigma}[\cdot])]. \quad (6)$$

q is the only hyperparameter introduced by the router; intuitively it is the per-layer fraction of tiles that are deemed risky enough to spend extra budget on. We use the same q across all layers and find it stable.

Choosing q . The working value $q = 0.40$ was selected from a sweep on RULER NIAH-multikey at $n = 30$, $\text{ctx}=32\text{K}$ (Appendix C), where paired recall rises monotonically with q over the tested range $\{0.10, 0.20, 0.30, 0.40\}$ and plateaus near 0.40. We use the same value unchanged across all benchmarks, models, and contexts in Section 4; the sweep was not repeated per task. A more thorough $\rho \times q$ Pareto study remains open (Section 4.7).

Why a quantile, not an absolute threshold. The absolute scale of σ varies across layers (early layers are more peaked than later layers). A quantile is the simplest layer-conditional normalisation. We did test absolute and per-cell quantile thresholds; they fail (Appendix C).

3.6 Step 6: expand kv_idx uniformly

Triggered tiles get a $\rho \times$ expanded top- k over tile_score (we use $\rho = 2$). The selection operation here is the same top- k that every selector in the literature reduces to; tile_score inherits whichever block-scoring backbone of §3 is in play (Eq. (1) or Eq. (2)), so the same expansion rule applies to both “router” and “router-on-Quest” without modification.

To preserve a fixed kernel dispatch shape across triggered and non-triggered tiles, we allocate $[B, H, Q_t, \rho k_{\text{budget}}]$ uniformly and pad non-triggered tiles’ extra slots with the kernel’s N_B -sentinel:

$$\text{kv_idx}[t, h] = \begin{cases} \text{top-}\rho k(\text{tile_score}[t, h, \cdot]) & \text{if trigger}[t] = 1, \\ \text{top-}k(\text{tile_score}[t, h, \cdot]) \parallel [N_B, \dots, N_B] & \text{otherwise.} \end{cases} \quad (7)$$

The kernel skips $b = N_B$ on a fast check, so non-triggered tiles cost the same as plain top- k . The end-to-end forward is otherwise unchanged: a standard FlashAttention-style online softmax, iterating each tile’s kv_idx instead of all N_B blocks.

Cost accounting. The average kept blocks per row is $k_{\text{budget}}(1 + q(\rho - 1))$. At $q = 0.4, \rho = 2$ this is $1.4 k_{\text{budget}}$. The measured wall-time cost is sub-proportional to this block-count factor: router/top- k kernel-time ratio is ≈ 1.53 at 32K and ≈ 1.47 at 64K (Section 4.6).

4 Experiments

4.1 Setup

Models. All weights frozen; no retraining, no fine-tuning. We evaluate on three instruction-tuned models spanning two architecture families:

- *Qwen2.5-14B-Instruct* – 40 Q / 8 KV heads, $d_{\text{head}}=128$, 48 layers, trained at 32K context. The headline model.
- *Qwen2.5-7B-Instruct-1M* – 28 Q / 4 KV heads, $d_{\text{head}}=128$, 28 layers, trained at 1M context. Same family as the headline model, but a smaller parameter count and a much longer training window; used for 128K-context experiments (Section 4.4) since the 14B model’s 32K training window precludes them.
- *Mistral-Nemo-Instruct-2407* – 32 Q / 8 KV heads, $d_{\text{head}}=128$, 40 layers, trained at 128K context. Different architecture family, used to test cross-family transfer. The Mistral tokenizer requires

`fix_mistral_regex=True` on `AutoTokenizer.from_pretrained`; otherwise the load path is unchanged.

The router code path and the v2 selection-plus-kernel implementation run unchanged across all three models; the only model-dependent code is the tokenizer flag above.

Hardware. A100-80GB via Modal. **Sparse path.** Custom Triton block-sparse attention kernel with fused per-tile selection. **Baselines and policies.**

- *dense* – FlashAttention (Dao et al., 2022) via PyTorch SDPA, the ceiling.
- *top-k* – per-tile top- k on the K-mean backbone (Eq. 1). SSA-style baseline; uses only Step 2.
- *Quest* – per-tile top- k on the Quest K-max upper-bound backbone (Eq. 2). Strong baseline on sharp-needle tasks.
- *router* – Steps 2–6 on top of the K-mean backbone, $q = 0.40, \rho = 2$. Uncertainty-gated expansion only.
- *router-on-Quest* – Steps 2–6 on top of the Quest K-max backbone, same q, ρ . Combines better scoring with budget allocation.

All four sparse policies share the same fused selection-plus-kernel implementation; they differ only in (a) which backbone scores blocks and (b) whether the router expansion is applied. Wall-time and quality numbers are therefore directly comparable across them. **Selector budget.** Fixed at $k_{\text{budget}} = 33$ blocks per tile, corresponding to roughly 2K attended tokens per row (the PCH diagnostic in Appendix B uses slightly different budgets, noted in-place).

Metrics. We report three quantities.

- **Accuracy** (or task score, e.g. F1): the standard unconditional success rate – fraction of examples the policy answers correctly. This is what the long-context literature usually reports.
- **Paired recall:** *among the examples dense answers correctly*, the fraction the sparse policy also answers correctly. We compute it as $|\{i : \text{dense}_i \text{ correct} \wedge \text{sparse}_i \text{ correct}\}| / |\{i : \text{dense}_i \text{ correct}\}|$. Dense paired recall is 1.0 by construction; a sparse policy with paired recall 1.0 would preserve every dense-correct example. We write n for the total number of examples run and $n_{dc} \leq n$ for the number of those examples dense answers correctly; n_{dc} **is the denominator of paired recall, not n** . The two coincide only when dense achieves 100% accuracy.
- **Hit rate:** fraction of gold-evidence blocks the selector keeps in `kv_idx`. Well-defined only when the gold blocks are known by construction; we use it for the diagnostic experiments in Appendix B, not for the headline benchmarks in this section.

Paired recall is not a standard named metric in the long-context literature – Quest, H2O, SnapKV, MInference, NSA, and SSA all report unconditional accuracy. We adopt it because raw accuracy conflates two failure modes: (i) the selector dropped a needed block, or (ii) the model could not have answered the question even with full attention. Paired conditions on $\{i : \text{dense}_i \text{ correct}\}$, which is the question set where (ii) is ruled out by construction; the gap between dense paired ($= 1.0$) and sparse paired is therefore attributable to the selector. This is the metric the experiments are built to move.

Note that paired recall does *not* imply accuracy ordering: a sparse policy can have lower paired recall and yet equal or higher raw accuracy than dense, simply by picking the right multiple-choice option for examples dense gets wrong (chance is 25% on 4-way MC; the sparse forward is a different, noisier predictor). We see exactly this on LongBench-v2 in Section 4.3.

4.2 RULER NIAH multi-key at $n = 100$, 32K context

Setup. RULER NIAH-multikey (3 keys) and VT (3 hops), $n_{\text{per}} = 50$ each ($n = 100$ total NIAH-multikey for the paired analysis), 32K context, $k_{\text{budget}} = 33$, $q = 0.40$, $\rho = 2$. Run independently on all three models; the harness, kernel, selection path, and tokenisation flags are model-agnostic (Section 4, Setup). **Result.**

Table 1: RULER NIAH-multikey paired recall at $n = 100$, $\text{ctx} = 32\text{K}$, across the three model panel. Paired = fraction of *dense-correct* examples the sparse policy also solves; n_{dc} is reported on the dense row and shared across the column. Backbone column indicates the block-scoring rule (K-mean = Eq. 1; K-max = Quest, Eq. 2). “–” indicates a configuration not run.

policy	backbone	Qwen-14B	Qwen-7B-1M	Nemo-12B
dense	–	1.00	1.00	1.00
n_{dc}	–	100	100	96
top- k	K-mean	0.51	0.28	0.29
router	K-mean	0.63	0.39	–
Quest	K-max	0.93	0.94	0.51
router-on-Quest	K-max	0.98	1.00	0.66
router-on-Quest – Quest		+0.05	+0.06	+0.15

Read. Three findings stack, all three reproduce across models:

1. **Router lifts paired recall on the K-mean backbone, but the baseline is fragile across models.** On Qwen-14B, $0.51 \rightarrow 0.63$ (+12 pp, $\approx 2.4\sigma$ at stderr 0.05, McNemar $p < 0.01$). On Qwen-7B-1M, $0.28 \rightarrow 0.39$ (+11 pp). The argument of the original method holds within the Qwen family; uncertainty-gated expansion preserves more dense-correct examples than plain top- k . The K-mean top- k baseline itself collapses on the smaller model and on Nemo (top- k drops to 0.28–0.29), so even where the router’s relative lift holds, it operates on a weak floor. The router-on-K-mean cell for Nemo was not run.
2. **Quest dominates K-mean within the Qwen family but does *not* saturate cross-family.** Within Qwen, Quest sits at 0.93–0.94 paired – a 1 pp delta between the two model sizes, consistent with the K-min/K-max upper bound being a robust scoring rule. On Mistral-Nemo, Quest paired drops to 0.51. The Quest backbone is the dominant lever, but *how well* it scores depends on the underlying attention distribution the model has been trained to produce; it is not a model-invariant constant. Mechanism: RULER NIAH needles are single hot keys; mean-pooling $\bar{k}_b$ dilutes them, so $q \cdot \bar{k}_b$ understates the block’s relevance, and Quest’s upper bound $\sum_d \max(q_d K_{b,d}^{\max}, q_d K_{b,d}^{\min})$ recovers it – to the extent the model produced sharp keys to begin with.
3. **Router-on-Quest is the headline composition, and the lift is regime-dependent.** On Qwen, router-on-Quest lifts paired recall by +5–+6 pp over Quest alone, landing within 0–2 pp of dense. On Nemo, where Quest has 49 pp of paired headroom rather than 6–7, the router lifts by **+15 pp** ($0.51 \rightarrow 0.66$). Under a one-sided McNemar bound (router-favouring discordant ≥ 14), this is significant at $p < 0.001$. The $3\times$ larger absolute lift on the model where Quest has room is direct evidence that the selectivity contribution of the router scales with the scoring-backbone headroom – statistically silent at the Qwen ceiling, statistically clean off it (see Table 4 below for the matched-budget decomposition on both models).

VT note. On RULER VT (3-hop) at 32K, dense itself fully solves only $\sim 5/100$ across all three models, so n_{dc} is statistically thin and the paired metric carries little information. Within that thin window, the

same stack-ordering holds on Qwen-14B: top- k 0.00, router 0.00, Quest 0.20, router-on-Quest **0.40**; the smaller models do not break the ordering but the absolute numbers are too small to report meaningfully. On unconditional partial recall (continuous, not paired), router-on-Quest on Qwen-14B scores 0.41 vs. dense 0.44 – nearly matching dense on a task where the K-mean baseline scores 0.14. VT at 3 hops is solved better by stronger base models; the selector picture should be re-evaluated when one is available.

4.3 LongBench-v2 medium, 32K context

Setup. LongBench-v2 *medium* split, multiple-choice (A/B/C/D) with paired threshold 1.0 (exact match). LB-v2 medium items are typically $\geq 100\text{K}$ words natively; we middle-truncate to 32K so the selector budget actually binds. Qwen-14B run at $n = 215$ (the entire medium-length subset of the dataset; no further filter); Qwen-7B-1M run at $n = 100$ (statistical-power cross-check; LB-v2 dense accuracy on the 7B model is roughly comparable to the 14B at 32K, so n_{dc} scales proportionally). Mistral-Nemo not run: a $n = 4$ smoke gave 0/4 dense-correct, projecting $n_{dc} \leq 10$ at $n = 100$ – insufficient power for a paired claim. Cross-family LB-v2 validation requires a stronger non-Qwen base model. **Result.**

Table 2: LongBench-v2 medium paired recall at $\text{ctx} = 32\text{K}$ across the model panel. “(n_{dc})” annotates the dense-correct denominator for each column. Backbone column as in Table 1.

policy	backbone	Qwen-14B ($n = 215$)	Qwen-7B-1M ($n = 100$)
dense	–	1.00	1.00
n_{dc}	–	40	15
top- k	K-mean	0.47	0.40
router	K-mean	0.60	0.47
Quest	K-max	0.68	0.60
router-on-Quest	K-max	0.75	0.80

Read. The same stacking pattern as RULER NIAH appears, with two refinements:

1. **Quest beats K-mean even on LB-v2 medium** (0.47 $\rightarrow$ 0.68 on Qwen-14B; 0.40 $\rightarrow$ 0.60 on Qwen-7B-1M). The Quest scoring backbone is therefore the dominant lever on *both* benchmarks, not just on RULER NIAH – including on a benchmark where evidence is distributed rather than concentrated in a single hot key.
2. **Router-on-Quest is the composition headline on both models.** Qwen-14B: paired 0.75 vs top- k 0.47 – +28 pp over the SSA-style baseline. With $n_{dc} = 40$ and $\text{stderr} \approx 0.10$ on the gap, this is $\approx 2.8\sigma$; McNemar’s exact test on the discordant pairs gives $p < 0.01$ – a statistically clean composition lift on a multi-hop real-task benchmark. Qwen-7B-1M reproduces directionally at $n_{dc} = 15$: router-on-Quest 0.80 vs top- k 0.40 – the absolute lift is +40 pp but the statistical power is thinner; the paired-recall ordering across all four sparse policies matches the Qwen-14B picture exactly.

Why all sparse accuracies sit at the same number despite very different paired recalls. Reading Table 2 naively obscures a metric pathology that paired recall is designed to surface. On Qwen-14B, every sparse policy posts *accuracy* in $[0.186, 0.209]$ while paired recall ranges over 0.47–0.75 – and Quest/router-on-Quest’s accuracy (0.20+) slightly exceeds dense (0.186). Decomposing router-on-Quest’s 44/215 correct answers: 30 come from the dense-correct set (paired $0.75 \times 40 = 30$); the remaining 14 come from dense-*wrong* examples where the sparse forward happened to pick the right MC option (14/175 on chance baseline 25% is below random, expected 44). Top- k ’s 41/215

decomposes as 19 paired hits (0.47×40) + 22 chance hits. Standard error on raw accuracy at this rate is $\sqrt{0.19 \cdot 0.81 / 215} \approx 0.027$, so the ≈ 0.02 spread across sparse policies is well inside noise. The selector-quality story lives entirely in the paired column.

4.4 Long-context quality scaling: 32K $\rightarrow$ 128K

The preceding subsections fix context at 32K and vary the model. Here we fix the model at Qwen2.5-7B-Instruct-1M (the only one of the three with a training window past 32K) and vary the context length. Same harness, same configuration ($k_{\text{budget}} = 33$, $q = 0.40$, $\rho = 2$), RULER NIAH-multikey at $n = 100$.

Table 3: RULER NIAH-multikey paired recall as context length grows, Qwen2.5-7B-Instruct-1M, $n = 100$. Dense is 100/100 at both 32K and 128K (within the model’s training window). All sparse policies degrade with context, but the K-mean-based policies (top- k , router-on-mean) collapse while Quest-based policies (Quest, router-on-Quest) hold most of their lift.

policy	32K paired	128K paired	Δ
top- k	0.28	0.09	−0.19
router (K-mean)	0.39	0.11	−0.28
Quest	0.94	0.77	−0.17
router-on-Quest	1.00	0.81	−0.19
dense	1.00	1.00	0

Read. The Quest backbone’s advantage *widens* with context: at 32K, Quest beats top- k by +66 pp on RULER NIAH paired; at 128K, by +68 pp. K-mean-based selection essentially breaks at 128K (top- k 0.09, router-on-mean 0.11) – the mean-pool signal is too dilute to recover single hot keys among 2,048 blocks. Router-on-Quest lands +4 pp over Quest alone at 128K and within 19 pp of dense, despite attending to only ≈ 92 blocks per tile ($\approx 4.5\%$ of context). Speed at 128K is reported separately in Section 4.6 alongside the 32K and 64K profiles.

4.5 Where the lift comes from: budget, selectivity, and when it activates

The headline results of Sections 4.2–4.4 raise two natural questions: is the router’s lift over Quest a genuine signal or a side effect of spending more budget, and does the lift hold on every long-context task? This subsection answers both, in that order.

Decomposing the lift: budget vs. selectivity. At $q = 0.40$ and $\rho = 2$, the router spends on average $\approx 1.4 \times$ Quest’s budget (92.4 effective blocks per tile, vs. 66 for Quest at $k_{\text{budget}} = 33$). Some of the lift over Quest could therefore come not from spending the marginal budget *selectively* – the router’s design intent – but simply from spending *more* budget on average. To separate the two contributions, we ran uniform Quest at a budget-matched control ($k_{\text{budget}} = 47$, effective 94) and a sweep of intermediate budgets on the same RULER NIAH-multikey examples used for Table 1, on both Qwen-14B and Mistral-Nemo.

Decomposition (Qwen-14B, near-ceiling). Of the +5 pp router-vs-Quest $_{k_{\text{budget}}=33}$ lift: $\approx +3$ pp comes from uniform budget expansion (Quest $_{k_{\text{budget}}=47}$ minus Quest $_{k_{\text{budget}}=33} = 0.96 - 0.93$), $\approx +2$ pp from selectivity (router minus Quest $_{k_{\text{budget}}=47} = 0.98 - 0.96$). Uniform Quest plateaus at 0.97 from

Table 4: Budget-match ablation on RULER NIAH-multikey, ctx=32K, $n = 100$. Effective budget includes the sink and self-block forced into every kv_idx; for the router it is the mean over tiles. Quest $_{k_{\text{budget}}=47}$ is the budget-matched control for the router at $q = 0.40$. n_{dc} is 100 on Qwen-14B and 96 on Mistral-Nemo (same dense-correct subsets as Table 1). The Qwen column is near the paired ceiling (Quest $_{k_{\text{budget}}=33} = 0.93$, dense 1.00); the Nemo column is off-ceiling (Quest $_{k_{\text{budget}}=33} = 0.51$).

policy	backbone	eff. budget	Qwen-14B paired	Nemo-12B paired
top- k	K-mean	66	0.51	0.29
Quest $k_{\text{budget}} = 33$	K-max	66	0.93	0.51
Quest $k_{\text{budget}} = 40$	K-max	80	0.95	0.53
Quest $k_{\text{budget}} = 47$ (matched)	K-max	94	0.96	0.57
Quest $k_{\text{budget}} = 52$	K-max	104	0.97	0.58
Quest $k_{\text{budget}} = 66$	K-max	132	0.97	0.60
router-on-Quest ($q = 0.40$)	K-max	92.4	0.98	0.66

$k_{\text{budget}} = 52$ onward; the router lands 1 pp above that asymptote, suggesting selectivity buys something genuinely orthogonal to raw budget. The Qwen RULER NIAH headline is therefore roughly **2/3 budget, 1/3 selectivity**. *Statistical caveat:* NIAH at 32K on Qwen-14B is near-ceiling. Dense is 1.00, Quest $_{k_{\text{budget}}=33}$ preserves 93/100, and the entire ablation lives in $[0.93, 0.98]$ – at most 7 examples of paired headroom. McNemar’s exact test on router vs Quest $_{k_{\text{budget}}=47}$ gives 3 discordant pairs router-favouring and 1 Quest-favouring, $p = 0.625$: directionally consistent, not separable by statistics at this n .

Decomposition (Mistral-Nemo, off-ceiling). Quest at $k_{\text{budget}} = 33$ on Nemo lands at 0.51 paired – 49 points of headroom rather than 7 – and the budget-vs-selectivity contributions are large enough to separate cleanly. Of the +15 pp router-vs-Quest $_{k_{\text{budget}}=33}$ lift: $\approx +6$ pp from uniform budget expansion (Quest $_{k_{\text{budget}}=47}$ minus Quest $_{k_{\text{budget}}=33} = 0.57 - 0.51$), $\approx +9$ pp from selectivity (router minus Quest $_{k_{\text{budget}}=47} = 0.66 - 0.57$). The Nemo split is **2/5 budget, 3/5 selectivity** – the inverse of the Qwen ratio. Two structural features sharpen the claim further: (i) uniform Quest does *not* plateau on Nemo within the budget range tested ($0.51 \rightarrow 0.53 \rightarrow 0.57 \rightarrow 0.58 \rightarrow 0.60$ as eff. grows $66 \rightarrow 132$); (ii) the router at eff. 92.4 beats uniform Quest $_{k_{\text{budget}}=66}$ at eff. 132 by +6 pp – a Pareto domination on the {paired, eff. budget} plane that no amount of uniform budget expansion within this range can close. *Statistics:* router preserves 63/96 dense-correct, Quest $_{k_{\text{budget}}=47}$ preserves 55/96; the gap is 8 examples. Under the most conservative reading (Quest $_{k_{\text{budget}}=47}$ ’s correct set is a subset of the router’s), discordant pairs favouring the router are ≥ 8 and against ≤ 0 ; the one-sided sign test gives $p \leq 0.5^8 \approx 0.004$. The actual McNemar count will be similar or larger in the favourable direction.

Reading the two ablations together. The fraction of the router’s lift attributable to selectivity is regime-dependent. Near the paired ceiling, the scoring backbone is doing almost all the work and the router’s role shrinks to filling a small remaining gap. Off the ceiling, the scoring backbone leaves substantial headroom and the router’s selective allocation captures most of it. This is the expected behaviour of a value-of-information policy: when uncertainty at the cutoff is rare (because scores are sharp), the router rarely triggers and budget expansion dominates the average lift; when uncertainty is common (because scores are diffuse), selective expansion at exactly those high-uncertainty cells dominates. The two ablations are not in tension – they bracket the regime in which the router operates.

When does the lift activate? LongBench-v1 as a negative control. The decomposition above answers *how* the router’s lift breaks down once it exists; it does not say *when* the lift exists at all. To pin that down we ran the LongBench-v1 panel (*musique-2hop*, *musique-4hop*, *narrativeqa*, *qasper*; $n = 30$, 32K context, same $k_{\text{budget}} = 33$, $q = 0.40$). LB-v1 native prompt lengths are 5–30K tokens; truncation to 32K is essentially a no-op, so the selector budget of 33 blocks ($\approx 2\text{K}$ attended tokens per row) is generous relative to typical needle density. Result: dense, top- k , and router essentially tie on F1 across all four tasks; paired ($F1 \geq 0.5$) shows top- k *already* preserves 100% of dense-correct examples on *musique-2hop*, *musique-4hop*, and *qasper* – no headroom for the router to recover. One small lift on *narrativeqa*: router paired 1.00 vs. top- k 0.89 (rescues 1/9), F1 0.307 vs. 0.276 (+3.1 pp). The combined LB-v1 / LB-v2 picture is the cleanest framing of when our method matters: **the router’s lift activates only when the per-query budget is small relative to where the answer-relevant evidence lives**. LB-v2 medium puts the budget under pressure (100K source $\rightarrow$ 32K window); LB-v1 does not. This is consistent with the budget-and-selectivity story above: the router can only re-allocate budget that is actually contested.

4.6 Efficiency: prefill speed from 32K to 128K

Claim. Router-on-Quest is slower than dense at 32K, faster at 64K, and substantially faster at 128K. The crossover is structural: attention’s share of prefill grows with context, and the rest-of-model floor is invariant across policies.

Setup. CUDA-event breakdown of one prefill forward (no decode) at $n = 8$. Decode ($Q = 1$) goes through SDPA dense for every policy, so policy-dependent wall time comes purely from prefill. 32K and 64K measurements are on Qwen2.5-14B-Instruct; 128K is on Qwen2.5-7B-Instruct-1M (the 14B model’s 32K training window precludes 128K). Both models use the same v2 kernel and the same $k_{\text{budget}} = 33$, $q = 0.40$; per-layer comparisons across the model-swap row are not directly meaningful, but the policy-vs-dense ratios on each row are.

Table 5: Prefill wall time across context lengths, $n = 8$. “vs. dense” is the wall-time ratio of the sparse policy to dense at the same context. 32K and 64K rows are on Qwen2.5-14B-Instruct; 128K row is on Qwen2.5-7B-Instruct-1M. Bold entries are the regimes where the sparse policy beats dense.

context	dense wall	top- k wall	router wall	top- k vs. dense	router vs. dense
32K	5,534 ms	5,349 ms	6,112 ms	0.97 $\times$	1.10 $\times$
64K	16,878 ms	12,747 ms	14,189 ms	0.76$\times$	0.84$\times$
128K	20,494 ms	11,769 ms	12,811 ms	0.57$\times$	0.62$\times$

Why 32K loses, why 128K wins (Amdahl). Attention’s share of dense prefill is 35% at 32K, 51% at 64K, 63% at 128K; the rest-of-model floor (linear layers, MLP, layernorms) is invariant across policies. Even an infinitely fast attention kernel can only speed up the attention share. At 32K the structural ceiling on speedup is $\approx 9\%$ (and the v2 kernel does not saturate it because selection adds back $\approx 7\%$); at 128K the ceiling is $\approx 63\%$ and the v2 kernel realises most of it (1.74 $\times$ predicted, 1.76 $\times$ observed for top- k). The 32K row is therefore a structural loss, not a kernel-engineering failure; the headline regime is 64K and beyond.

Kernel and selection scaling. The v2 kernel scales sub-linearly with N (per-layer time scales $\approx 1.4\times$ per doubling of context, dominated by HBM I/O at long context); SDPA scales quadratically ($\approx 4\times$ per doubling). The cross-over is therefore not a coincidence – it is what the kernel’s design predicts. Selection (the $q \cdot \bar{K}^T$ pool-product plus topk) currently scales $\approx 2\times$ per doubling and takes 7% / 12% / 16% of prefill at 32K / 64K / 128K. By 256K it would rival the kernel itself; caching $\bar{K}$ once per layer

and fusing selection into the kernel’s first pass are the natural next-step optimisations, but they are engineering work and do not change the method.

Router vs. top- k . The router pays an extra $\approx 1.5\times$ kernel time per triggered tile because it expands $k_{\text{budget}} \rightarrow \rho k_{\text{budget}}$ on the bottom- q fraction (Section 3, Step 6); this shows up as the $0.05\text{--}0.08\times$ gap between the top- k and router columns at 64K and 128K. The router’s quality lift (Section 4.2–4.4) buys this back: at 128K, router-on-Quest holds paired recall 0.81 vs. dense 1.00 at $0.62\times$ dense wall time.

4.7 Limitations

1. **Per-tile granularity.** Per-tile selection structurally loses needles that one row strongly wants but other rows in the tile ignore. The router recovers part of this gap but not all. A per-row scatter-into-bitmap kernel would close the rest of the gap without re-materialising the $[B, H, M, N_B]$ keep tensor; not yet built.
2. **Cross-family coverage at 32K, but constrained at 128K.** The model panel (Section 4, Setup) covers two families and three models, validating at 32K that the composition lift and the budget-vs-selectivity decomposition (Section 4.5) hold on a non-Qwen architecture. Two transfer gaps remain. First, on real-task QA: LongBench-v2 on Mistral-Nemo was attempted but the model’s hard-multiple-choice base accuracy at 32K is too low to yield a usable n_{dc} (0/4 dense-correct in a smoke), so cross-family LB-v2 requires a stronger non-Qwen model – Llama-3.1-8B-Instruct is the natural candidate but requires HF gating; Mistral-Small-3.1-24B is the next Mistral size up but also gated. Second, at long context: Mistral-Nemo at 128K OOMs on A100-80GB (≈ 24 GB weights + ≈ 11 GB KV cache at 128K + activations), so the only 128K data point is Qwen2.5-7B-Instruct-1M (Section 4.4). Higher-memory hardware (H100-94GB, H200) or a chunked-prefill harness would unblock cross-family 128K; both are engineering work, not method changes. The remaining open standardised benchmark is HELMET-RAG.
3. **Hyperparameter sweeps incomplete.** The trigger fraction $q = 0.40$ was selected from a RULER NIAH-multikey sweep at $n = 30$ and used unchanged on every benchmark, model, and context length in the paper (Appendix C). We did not re-sweep q per task; LongBench-v2 medium in particular plausibly benefits from a different value because the budget-vs-evidence Pareto shifts (distributed evidence vs. single hot keys). The expansion factor $\rho = 2$ is similarly fixed throughout; a $\rho \in \{1.25, 1.5, 2\}$ Pareto sweep would characterise the speed/quality trade-off the router exposes, but is not yet run.
4. **What’s bounded, what’s owed.** Appendix D anchors the σ -quantile router as the dimensionless analogue of the asymptotically optimal best-arm-identification (BAI) exploration index of Garivier and Kaufmann (2016), with a $C(\tau)$ -times-optimal statement for top- k *identification* under sub-Gaussian noise on block scores. The remaining gap is from *identification* to *attention output*: the downstream quantity we actually care about is $\|o^{\tilde{S}} - o^{\text{full}}\|$, which depends on the softmax mass on dropped blocks, not their identity. Composing the BAI bound with an attention-output sensitivity bound – a $C(\tau) \cdot f(\sigma)$ -times-optimal statement on the output itself – is the natural follow-up theorem. This tightens what the BAI bound already delivers; it is not strictly needed for the empirical claims.

Conclusion

In this work we presented an uncertainty-gated value-of-information *router* for block-sparse attention selectors: a per-tile cutoff-margin signal σ triggers a $\rho\times$ expansion of the kept set on the bottom q -

fraction of tiles per layer, independent of how block scores are computed and stackable with existing scoring backbones such as Quest.

On standardised long-context benchmarks the router delivers measurable quality gains. On LongBench-v2 medium ($n = 215, 32K$) router-on-Quest paired recall reaches 0.75 vs. top- k 0.47 – a statistically clean composition lift (+28 pp, McNemar $p < 0.01$) on a multi-hop real-task benchmark. The lift reproduces on three models from two architecture families: on Mistral-Nemo-Instruct-2407, where Quest has paired headroom rather than saturating, the router beats Quest by +15 pp ($p < 0.001$). At 128K context, router-on-Quest holds paired recall 0.81 while the SSA-style baseline collapses to 0.09, and the fused selection-plus-kernel pipeline runs at $0.62\times$ dense wall time – beating dense from 64K onward as predicted by Amdahl arithmetic.

We are excited to extend this line of work in three directions. The most immediate empirical gaps are third-family validation beyond Qwen and Mistral (Llama-3, MiniMax) and a stronger non-Qwen base model for cross-family LongBench-v2; HELMET-RAG would broaden the standardised benchmark base. The most immediate engineering gain is fusing block selection into the attention kernel, which the per-layer scaling profile projects to become the dominant cost at 256K+. The BAI bound in Appendix D characterises top- k identification under noisy block scores; composing it with an attention-output sensitivity bound is the natural follow-up theorem.

Code, results, and reproduction scripts: <https://github.com/ThomasRossi/uncertainty-gated-block-sparse-a>

A Step-by-step derivation of σ and the trigger rule

This appendix walks through Section 3 at one equation per step, so a reader who has not followed the body can reproduce the implementation. Notation: B batch, H heads, M query rows, N keys, $N_B = N/\text{BLOCK}_N$ key blocks, $Q_t = M/\text{BLOCK}_M$ query tiles, $d_{\text{head}} = d_{\text{model}}/H$.

(A.1) Block-pooled keys.

$$\bar{k}_b^{(L,h)} = \frac{1}{\text{BLOCK}_N} \sum_{j \in \text{block } b} k_j^{(L,h)}, \quad b \in \{0, \dots, N_B - 1\}. \quad (8)$$

(A.2) Per-row block scores.

$$s^{(L)}[i, h, b] = \frac{q_i^{(L,h)} \cdot \bar{k}_b^{(L,h)}}{\sqrt{d_{\text{head}}}}, \quad i \in \{0, \dots, M - 1\}. \quad (9)$$

(A.3) Per-tile reduction. For Q-tile t covering rows $\{t\text{BLOCK}_M, \dots, (t+1)\text{BLOCK}_M - 1\}$,

$$\text{tile_score}[t, h, b] = \max_{r \in \text{tile } t} s[r, h, b]. \quad (10)$$

We use max-pool because dropping a block the most aggressive row wants is worse than dropping one only weak rows want; mean-pool was tested and underperforms.

(A.4) Forcing the sink and self-block. Let $b_{\text{sink}} = 0$ and $b_{\text{self}}(t) = \lfloor t \text{BLOCK}_M / \text{BLOCK}_N \rfloor$ (the key block containing the tile’s own rows). We add $+\infty$ to the corresponding entries of tile_score before the top- k :

$$\widetilde{\text{tile_score}}[t, h, b] = \text{tile_score}[t, h, b] + \infty \cdot \mathbb{I}[b \in \{b_{\text{sink}}, b_{\text{self}}(t)\}]. \quad (11)$$

This guarantees both blocks land in the kept set without a separate concat-and-dedup.

(A.5) Top- $(k+1)$ and the sorted prefix. We compute

$$\text{idx}_{\text{sorted}}[t, h, \cdot], \quad \widetilde{s}_{\text{sorted}}[t, h, \cdot] = \text{top-}(k+1)(\widetilde{\text{tile_score}}[t, h, \cdot]), \quad (12)$$

in descending order. The first k indices are $\text{kv_idx}[t, h]$; the $(k+1)$ -th score is what we need for σ .

(A.6) Normalised cutoff margin.

$$\sigma[t, h] = \frac{\widetilde{s}_{\text{sorted}}[t, h, k-1] - \widetilde{s}_{\text{sorted}}[t, h, k]}{\widetilde{s}_{\text{sorted}}[t, h, 0] - \widetilde{s}_{\text{sorted}}[t, h, k]}. \quad (13)$$

Edge cases: if the denominator is zero (all top scores tied) we set $\sigma = 0$ (treat as maximally uncertain); $+\infty$ entries from (A.4) participate in the sorted prefix on top, so $\widetilde{s}_{\text{sorted}}[t, h, 0]$ is always $+\infty$ – in practice we therefore exclude the forced indices from the σ computation and apply Eq. (A.6) to the remaining tile scores.

(A.7) Head aggregation.

$$\bar{\sigma}[t] = \frac{1}{H} \sum_{h=1}^H \sigma[t, h]. \quad (14)$$

(A.8) Per-layer empirical quantile.

$$\tau^{(L)} = \text{quantile}_q(\bar{\sigma}[\cdot]), \quad \text{trigger}[t] = \mathbb{I}[\bar{\sigma}[t] \leq \tau^{(L)}]. \quad (15)$$

(A.9) Padded expansion. Working budget k_{budget} , expansion factor ρ , sentinel $b_{\perp} = N_B$. We allocate $\text{kv_idx}_{\text{final}} \in \mathbb{Z}^{B \times H \times Q_t \times \rho k_{\text{budget}}}$ and fill

$$\text{kv_idx}_{\text{final}}[t, h] = \begin{cases} [\text{idx}_{\text{sorted}}[t, h, 0], \dots, \text{idx}_{\text{sorted}}[t, h, \rho k - 1]] & \text{trigger}[t] = 1, \\ [\text{idx}_{\text{sorted}}[t, h, 0], \dots, \text{idx}_{\text{sorted}}[t, h, k - 1], b_{\perp}, \dots, b_{\perp}] & \text{otherwise.} \end{cases} \quad (16)$$

The triggered case requires a top- ρk rather than a top- $(k+1)$, which we batch with the top- $(k+1)$ used for σ as a single top-max($\rho k, k+1$) = ρk when $\rho \geq 2$.

(A.10) Kernel iteration. The Triton kernel’s inner loop iterates $\text{kv_idx}_{\text{final}}[t, h]$ and skips any entry equal to $b_{\perp}$ on a single compare. The online softmax is unchanged from FlashAttention.

Average attended budget. A row in tile t attends to k blocks if $\text{trigger}[t] = 0$ and ρk if $\text{trigger}[t] = 1$. The expected per-row attended budget is therefore

$$\mathbb{E}[\text{attended}] = k_{\text{budget}} \cdot (1 + q(\rho - 1)), \quad (17)$$

which at $q = 0.4, \rho = 2$ is $1.4 k_{\text{budget}}$. This is the headline cost of the router.

B Pointer-Chase Haystack: construction and diagnostic results

The Pointer-Chase Haystack (PCH) is a custom diagnostic benchmark we built during method development to isolate selector quality from model capability. We retain it here for transparency on the development process; the headline empirical results in Section 4 use the standardised benchmarks RULER NIAH and LongBench-v2 medium.

Design goals. We needed a task that (i) has a dense-attention ceiling of $\approx 100\%$, so failures are unambiguously selector failures, and (ii) is *query-latent*, meaning the relevant tokens cannot be identified by surface matching against the query alone. RULER NIAH multi-key satisfies (i) but not (ii); RULER VT satisfies (ii) but not (i) (Qwen 7B fails it).

Construction. A pointer chase of depth h is a sequence of indexed entries

Entry ID₀: continue at entry ID₁.
Entry ID₁: continue at entry ID₂.
 $\vdots$
Entry ID_h: the recorded value is V.

with all ID _{k} drawn from a private namespace. A PCH instance contains one gold chain plus D distractor chains, all entries shuffled into a haystack of target token length N . The query gives ID₀ and asks for V . Hop 0 is query-identifiable; hops $1, \dots, h$ are query-latent. Compounding is by construction: missing entry k permanently blocks every entry $k+1, \dots, h$.

Grounding. The pointer chase is the canonical communication-complexity problem for unavoidable k -round sequential dependency: it provably cannot be collapsed into fewer rounds. Every per-hop operation is trivial (follow a link, read a value), keeping a capable dense model at the $\sim 100\%$ ceiling.

Metric. We report *hit rate*: the fraction of gold-chain blocks the selector keeps in kv_idx. Hit rate is well-defined only when the gold blocks are known by construction, which is the case here.

Result. Table 6 reports hit rate at $h = 3$, fixed $k_{\text{budget}} = 40$ blocks/row (32K, 64K) and 29 (8K), on Qwen2.5-14B-Instruct.

Table 6: PCH hop=3 hit rate. Hit rate is the fraction of gold-chain blocks the selector keeps. Steady-state wall time is reported for orientation; the main efficiency analysis is in Section 4.6.

ctx	policy	dt (steady)	vs dense	hit rate
8K	dense	1.4s	1.00×	1.00
8K	top- k (per-tile)	1.6s	1.14× slower	0.79
8K	router q=0.10	1.6s	1.14× slower	1.00
32K	dense	6.9s	1.00×	1.00
32K	top- k	6.05s	0.88× (12% faster)	0.63
32K	router q=0.10	6.6s	0.96× (4% faster)	0.78
64K	dense	17.5s	1.00×	1.00
64K	top- k	14.1s	0.81× (1.24× faster)	0.42
64K	router q=0.10	14.8s	0.85× (1.18× faster)	0.54

Router lifts hit rate over plain top- k by +12–+15 pp across scales (and recovers dense’s 1.00 at 8K). The gain is monotonic with context length, consistent with the structural cost of per-tile selection increasing with N that motivates the router in the first place.

C Per-row vs. per-cell aggregation: why heads are pooled before thresholding

We tested two natural alternatives to Eq. (6) and they both fail. We report them here so the design choice in Section 3 can be understood as a tested decision rather than an arbitrary one.

Per-cell quantile. Trigger on $\mathbb{K}[\sigma[t, h] \leq \text{quantile}_q(\sigma[\cdot, \cdot])]$, i.e. flag the bottom- q *cells*, not tiles. Result: the rescue set is dominated by per-head idiosyncratic noise; the gain over plain top- k vanishes for every q we tested. Mechanism: heads in the same tile are highly correlated (PCH cross-head agreement matches peakedness byte-for-byte across candidates), so per-head σ values carry mostly the same signal plus head-private noise; quantile-by-cell amplifies the noise.

Absolute threshold τ on σ . Trigger on $\mathbb{K}[\sigma[t, h] \leq \tau]$ for a fixed τ . Result: works at one layer’s σ distribution but not across the stack; we observe systematic distribution shift early-vs-late layers. A per-layer quantile is the simplest layer-conditional fix and dominated the absolute- τ sweep.

Row-grain q sweep. On PCH at 32K, row-grain $q = 0.10$ reaches near-top- k effective budget at hit rate 0.78, vs. plain top- k at 0.63. On RULER NIAH-multikey at $n = 30$, ctx=32K (fixed $k_{\text{budget}} = 33$, kernel-v2, row-grain quantile router), we swept $q \in \{0.10, 0.20, 0.30, 0.40\}$ and observed paired recall 0.38, 0.35, 0.47, 0.53 respectively, vs. top- k baseline 0.35. The trend is monotonic in q over the tested range and plateaus near $q = 0.40$. We use $q = 0.40$ for the headline results on RULER and LB-v2; the regime where q should differ across tasks (because the budget-vs-evidence Pareto shifts) is consistent with the smaller value chosen for PCH, but a per-task sweep on LB-v2 medium remains open.

D σ as a best-arm-identification exploration index

The Method section introduces σ as a value-of-information proxy and validates it empirically. This appendix gives the formal anchor: under a noisy-score model on the per-block relevance signal, σ is a dimensionless analogue of the exploration index used by the asymptotically optimal top- k best-arm identification (BAI) algorithm of [Garivier and Kaufmann \(2016\)](#). The argument lets us state how far the σ -router can be from optimal under a budget constraint – a $C(\tau)$ -times-optimal statement under sub-Gaussian noise – and clarifies what the bound does *not* cover.

D.1 Setting: top- k block selection as a BAI problem

Fix a Q-tile t and head h (we suppress both for readability). For each block $b \in \{0, \dots, N_B - 1\}$, the per-tile score is

$$s_b = \mu_b + \eta_b, \quad \eta_b \text{ iid sub-Gaussian with scale parameter } \tau, \quad (18)$$

where μ_b is the true (unobservable) relevance of block b to the rows of tile t , in the sense of contribution to attention output. We assume μ has distinct top- k structure: there is a well-defined set $S^*(\mu) \subset \{0, \dots, N_B - 1\}$ of size k such that $\min_{b \in S^*} \mu_b > \max_{b \notin S^*} \mu_b$. Under the budget constraint that we may attend to at most K blocks across the tile per layer (where $K = k_{\text{budget}}$ for non-triggered tiles and $K = \rho k_{\text{budget}}$ for triggered ones), the question is: which subset $\hat{S}$ of size $\leq K$ should we pick?

The classical top- k identification problem asks for $\hat{S}$ such that $\mathbb{P}[\hat{S} \neq S^*(\mu)] \leq \delta$ using as few "samples" as possible. In our setting, "samples" become budget: every block we include in $\hat{S}$ is one we attend to. The mapping is exact when $K = N_B$ (full attention, no budget pressure) and increasingly lossy as K shrinks.

D.2 The information-theoretic lower bound

Kaufmann et al. (2016) and Garivier and Kaufmann (2016) characterise the minimum expected sample complexity of δ -correct top- k identification under (18). For any algorithm $\mathcal{A}$ returning $\hat{S}_{\mathcal{A}}$ with $\mathbb{P}[\hat{S}_{\mathcal{A}} \neq S^*(\mu)] \leq \delta$ uniformly over μ ,

$$\mathbb{E}_{\mu}[T_{\mathcal{A}}] \geq T^*(\mu) \cdot \log(1/\delta) + o(\log(1/\delta)), \quad \delta \rightarrow 0, \quad (19)$$

where $T^*(\mu)$ is the optimal value of a max-min programme:

$$T^*(\mu)^{-1} = \max_{w \in \Delta_{N_B}} \min_{\nu: S^*(\nu) \neq S^*(\mu)} \sum_{b=1}^{N_B} w_b \cdot d(\mu_b, \nu_b), \quad (20)$$

with $d(\cdot, \cdot)$ the KL divergence between the two noise models and Δ_{N_B} the simplex of allocations. The minimiser ν in (20) corresponds to the hardest "alternative" world in which the top- k is different from μ 's – typically achieved by perturbing the two arms at the cutoff boundary, $(k-1)$ and (k) .

D.3 Track-and-Stop and the exploration index at the cutoff

The Track-and-Stop algorithm of Garivier and Kaufmann (2016) matches the lower bound (19) asymptotically. After observing $\hat{\mu}_b$, its next-sample rule is

$$\text{pull arm } b^* = \arg \max_b [w_b^*(\hat{\mu}) - T_b/T], \quad (21)$$

where w^* solves (20) at $\hat{\mu}$ and T_b/T is the empirical allocation so far. Under our top- k structural assumption, the optimal allocation w^* concentrates on the cutoff pair: arms $(k-1)$ and (k) in the sorted order receive most of the next sample. Equivalently, the per-arm "value of next sample" near the cutoff is monotone in

$$V_{\text{TaS}}(\hat{\mu}) \propto (\hat{\mu}_{(k-1)} - \hat{\mu}_{(k)})^{-2}, \quad (22)$$

under sub-Gaussian noise (standard BAI calculation: KL divergence between Gaussians of equal variance scales as the squared mean gap, so the value of distinguishing them scales inversely). **Small empirical gap at the cutoff $\iff$ high exploration value $\iff$ allocate more budget here.**

D.4 σ as a dimensionless Track-and-Stop index

The router we use does not estimate τ . Track-and-Stop, by contrast, requires τ to scale (22) to absolute units. To dispense with τ , we normalise the cutoff gap by an empirical proxy for the score spread:

$$\sigma = \frac{s_{(k-1)} - s_{(k)}}{s_{(0)} - s_{(k)}}. \quad (23)$$

Two observations.

σ preserves the cutoff-ordering of Track-and-Stop. On a single tile, both numerator (cutoff gap) and denominator (score spread) of (23) are deterministic given s ; ranking tiles by σ ranks them by an order-preserving transform of the cutoff gap, with the spread acting as a per-tile scale normaliser. Since Track-and-Stop’s per-step decision is “which arm pair has the smallest gap relative to noise scale,” and we use $s_{(0)} - s_{(k)}$ as a tile-level proxy for that noise scale, the relative ordering of tiles by exploration value under (22) is preserved by σ^{-1} . Bottom- q thresholding on σ therefore implements the same “allocate to the most-disputed cutoffs” rule as Track-and-Stop, at the granularity of tiles rather than individual arm pulls.

The denominator is the right scale. An absolute-threshold rule (Appendix C) fails because the marginal distribution of s shifts across layers; the per-tile spread $s_{(0)} - s_{(k)}$ is the simplest layer-conditional and tile-conditional normaliser that makes σ dimensionless and comparable. Under (18) the spread concentrates around its mean as $N_B \rightarrow \infty$, so $s_{(0)} - s_{(k)}$ is also a consistent estimator of the relevant scale.

D.5 Suboptimality bound

Proposition 1 (Informal). *Under (18) with sub-Gaussian noise of scale τ , the σ -quantile router achieves expected top- k identification error after total budget T satisfying*

$$\mathcal{E}_\sigma(T) \leq C(\tau) \cdot \mathcal{E}^*(T) + o(1) \quad \text{as } T \rightarrow \infty, \quad (24)$$

where $\mathcal{E}^*(T)$ is the error of Track-and-Stop at the same budget and $C(\tau)$ depends only on the noise scale. For Gaussian noise, $C(\tau) \leq 2$; for general sub-Gaussian, $C(\tau)$ scales linearly in the ratio of sub-Gaussian to true variance.

Proof sketch. The Track-and-Stop allocation w_t^* for tile t depends on the absolute gap $\hat{\mu}_{(k-1)}^{(t)} - \hat{\mu}_{(k)}^{(t)}$ scaled by τ . The σ -quantile policy uses $\sigma_t = (\hat{\mu}_{(k-1)}^{(t)} - \hat{\mu}_{(k)}^{(t)}) / (\hat{\mu}_{(0)}^{(t)} - \hat{\mu}_{(k)}^{(t)})$. Two sources of suboptimality:

1. *Spread vs. noise mismatch.* We use $\hat{\mu}_{(0)} - \hat{\mu}_{(k)}$ as a scale instead of τ . Under (18), $\hat{\mu}_{(0)} - \hat{\mu}_{(k)} = (\mu_{(0)} - \mu_{(k)}) \cdot (1 + O(\tau/\mu_{(0)}))$, so the scale is correct up to a factor $(\mu_{(0)} - \mu_{(k)})/\tau$. The error this introduces in the per-tile ordering of exploration values is captured by the constant $C(\tau)$. For Gaussian noise the factor is ≤ 2 by a direct computation; for general sub-Gaussian, it scales linearly with the variance proxy.
2. *Quantile vs. continuous allocation.* Track-and-Stop allocates a continuous fraction of budget to each arm; we make a binary expand/don’t-expand decision. This introduces a one-step discretisation cost that decays as $1/\sqrt{T}$ and is absorbed into the $o(1)$ term as $T \rightarrow \infty$.

A careful version of this sketch substitutes the empirical-process bound of Garivier and Kaufmann (2016) for the discretisation step. $\square$

What (24) does *not* bound. The proposition controls top- k identification error – the probability that the kept set $\hat{S}$ misses an element of $S^*(\mu)$. The quantity we care about for downstream task quality is the *attention-output* error,

$$\|o^{\hat{S}} - o^{\text{full}}\|_2,$$

which depends on the softmax mass on dropped blocks rather than on the identity of those blocks. The two are related but not equivalent: a mis-identified $\hat{S}$ in which the missed block carries negligible softmax

mass costs nothing in output error. Composing (24) with an attention-output sensitivity bound (left for future work; sketched in the discussion accompanying this paper) would give the natural composite statement: *the σ -router’s attention output is within $C(\tau) \cdot f(\sigma) \cdot \max_j \|v_j\|$ of the optimal-sparse output under the noise model (18).*

D.6 Empirical corroboration

The bound (24) is asymptotic; we cannot directly read $C(\tau)$ off the experiments. The closest empirical proxy is the gap between the router’s paired recall and the dense ceiling on a benchmark where dense achieves identification with high accuracy:

- **RULER NIAH-multikey**, $n = 100$: dense paired 1.00; router-on-Quest paired 0.98. The router lands within 2 pp of the infinite-budget ceiling at $k_{\text{budget}} = 33$. Under (18) with sharp needles (low effective τ relative to $\mu_{(0)} - \mu_{(k)}$), the bound predicts $C(\tau)$ close to 1, i.e. the router is close to optimal at this budget. The empirical 2-pp gap is consistent with this regime.
- **LB-v2 medium**, $n = 100$: dense paired 1.00; router-on-Quest paired 0.75. A 25-pp gap. Two interpretations remain compatible with the bound: (i) the noise model (18) fits poorly (distributed evidence, no single hot μ), pushing $C(\tau)$ large, or (ii) the budget $k_{\text{budget}} = 33$ is genuinely insufficient and no policy at this budget could reach 1.00. Distinguishing these requires a budget-vs-paired-recall sweep, which is open work (Section 4.7).

D.7 What’s bought, what’s owed

Bought. A formal anchor: σ is the dimensionless analogue of the asymptotically optimal BAI exploration index under sub-Gaussian noise on block scores. The σ -quantile router is $C(\tau)$ -times optimal in top- k identification, with $C(\tau) \leq 2$ under Gaussian noise. The empirical 2-pp gap to dense on RULER NIAH corroborates this in the sharp-needle regime.

Owed. The composite attention-output bound; an explicit constant for the noise models that fit our actual block scores (empirical fit of τ , possibly per-layer); and a treatment of the LB-v2 medium gap that distinguishes “the bound is loose here” from “the budget is binding here.”

References

- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017. <https://arxiv.org/abs/1706.03762>.
- J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *MLSys*, 2024. <https://arxiv.org/abs/2406.10774>.
- Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2023. <https://arxiv.org/abs/2306.14048>.
- Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen. SnapKV: LLM knows what you are looking for before generation. In *NeurIPS*, 2024. <https://arxiv.org/abs/2404.14469>.

- H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *NeurIPS*, 2024. <https://arxiv.org/abs/2407.02490>.
- J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. X. Wei, L. Wang, Z. Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. arXiv preprint, 2025. <https://arxiv.org/abs/2502.11089>.
- E. Lu, Z. Jiang, J. Liu, Y. Du, T. Jiang, C. Hong, S. Liu, W. He, E. Yuan, Y. Wang, et al. MoBA: Mixture of block attention for long-context LLMs. arXiv preprint, 2025. <https://arxiv.org/abs/2502.13189>.
- Subquadratic. How SSA makes long-context practical. 2025. <https://subq.ai/how-ssa-makes-long-context-practical>.
- T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022. <https://arxiv.org/abs/2205.14135>.
- C.-P. Hsieh, S. Sun, S. Krizan, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. RULER: What’s the real context size of your long-context language models? In *COLM*, 2024. <https://arxiv.org/abs/2404.06654>.
- Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *ACL*, 2024. <https://arxiv.org/abs/2308.14508>.
- Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. arXiv preprint, 2024. <https://arxiv.org/abs/2412.15204>.
- H. Yen, T. Gao, M. Hou, K. Ding, D. Fleischer, P. Izsak, M. Wasserblat, and D. Chen. HELMET: How to evaluate long-context language models effectively and thoroughly. arXiv preprint, 2024. <https://arxiv.org/abs/2410.02694>.
- A. Garivier and E. Kaufmann. Optimal best arm identification with fixed confidence. In *COLT*, 2016.
- E. Kaufmann, O. Cappé, and A. Garivier. On the complexity of best-arm identification in multi-armed bandit models. *JMLR*, 17(1):1–42, 2016.